Übungsblatt 6: Server-Verfahren für ereignisgetriebene Last

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: CPU-Budget und periodische Interferenz (8 Punkte)

Alle Werte sind Lehrannahmen in Millisekunden. Ein Prozessor, feste Prioritäten, vollständige Präemption, keine Sperren, Selbstsuspendierung oder Zusatzkosten. Periodische Jobs werden synchron bei null freigegeben, ihre Deadline entspricht der Periode. Der Polling Server wird für diesen Nachweis konservativ bei jeder Freigabe mit seinem vollen Budget berücksichtigt.

Pfad	CPU-Zeit / Budget	Periode	Priorität
Regelung	2	7	höchste
Polling Server	2	14	mittlere
Zustandsmeldung	3	28	niedrigste

Verwenden Sie $R_i^{(0)} = C_i$ und

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j.$$

Für den Server gilt in dieser Rechnung $C_S = Q_S$. $hp(i)$ enthält die höher priorisierten Pfade.

1. In einem getrennten Lauf rechnet der Server von 1 bis 2 und von 5 bis 6 ms. Welches Budget verbraucht er, welches Intervall liegt zwischen erstem Start und Ende und wie viel bleibt von 2 ms Budget? (2 Punkte)
2. Berechnen Sie die Gesamtauslastung des Tabellenmodells. (2 Punkte)
3. Bestimmen Sie die Antwortzeitfixpunkte für Server und Zustandsmeldung. (3 Punkte)
4. Bewerten Sie deren periodische Modellfristen. Ist die Serverantwort automatisch die Antwort jeder bei ihm eingereihten Anfrage? (1 Punkt)

Aufgabe 2: Ein Polling Server mit unterbrochener Anfrage (8 Punkte)

Ein Server hat $Q_S = 3$ ms und $T_S = 12$ ms. Keine andere Arbeit, keine Zusatzkosten, FIFO-Reihenfolge. Die logischen Pollzeitpunkte sind 0, 12, 24, … ms. Neue Ankünfte am Pollzeitpunkt zählen vor der Pollprüfung. Bei leerem Poll verfällt das gesamte Budget. Wird die Arbeit später vollständig leer, verfällt auch Restbudget. Eine angefangene Anfrage darf bei Budgetende unterbrochen werden; ihr Rest wird vor jüngeren Anfragen fortgesetzt. Es gibt genug Pufferplatz.

Anfrage	Ankunft	eigene CPU-Zeit
A	2	1
B	5	1
C	11	2
D	13	1

1. Zeichnen Sie Serverausführung und Leerlauf für $[0, 27)$ ms. (3 Punkte)
2. Bestimmen Sie Abschluss und Antwortzeit jeder Anfrage. (4 Punkte)
3. Wie viel Restbudget bleibt nach dem letzten Abschluss und was geschieht damit? (1 Punkt)

Aufgabe 3: Deferrable und Sporadic Server vergleichen (8 Punkte)

Für beide Varianten gelten $Q_S = 3$ ms, $T_S = 10$ ms und anfangs drei Budgeteinheiten. Keine anderen Tasks, keine Zusatzkosten, keine Unterbrechungen außer Budgeterschöpfung, FIFO. A kommt bei 7 ms mit 3 ms Bedarf, B bei 10 ms mit 3 ms Bedarf. Bei gleichen Zeitpunkten werden Ankünfte und fällige Auffüllungen vor der Auswahl berücksichtigt.

Der **Deferrable Server** setzt bei 0, 10, 20, … sein Budget auf drei zurück; nicht verbrauchtes altes Budget wird dabei nicht addiert.

Für den **Sporadic Server** betrachten wir nur die hier eindeutigen aktiven Abschnitte ohne höher priorisierte Arbeit: Ein aktiver Abschnitt beginnt beim Serverstart t_a. Verbraucht er bis zum Leerwerden oder zur Budgeterschöpfung q, wird diese Menge bei $t_a + T_S$ zurückgegeben. Im Leerlauf bleibt ungenutztes Budget erhalten. Es gibt keinen pauschalen Reset bei 10 ms. Überlappende Auffüllungen während eines aktiven Abschnitts treten in dieser Aufgabe nicht auf.

1. Geben Sie beide Ausführungsintervalle des Deferrable Servers an. (2 Punkte)
2. Geben Sie beim Sporadic Server die Intervalle, die beiden Auffülltermine mit Mengen und die Antwortzeit von B an. (4 Punkte)
3. Wie viel CPU-Zeit beansprucht der Deferrable Server in $[7, 13)$? Warum kann man aus Q_S/T_S allein keinen Fristnachweis für niedrigere Tasks ableiten? (2 Punkte)

Aufgabe 4: Überlast und Grenzen einer Implementierung (8 Punkte)

Ein Server bietet $Q_S = 2$ ms je $T_S = 10$ ms. Jede Anfrage braucht genau 0,5 ms CPU-Zeit. Nehmen Sie für die Ratenrechnung an, dass jedes Budget vollständig ausgeführt werden kann und genügend Arbeit vorliegt. Kosten und Auslassungen fehlen zunächst.

1. Wie viele solche Anfragen passen in ein Budget, welche langfristige Bedienrate ergibt sich? (2 Punkte)
2. Es kommen langfristig 500 Anfragen/s. Bestimmen Sie den Belegungszuwachs. Ein **vereinfachtes stetiges Ratenmodell** beginnt mit 20 freien Queueplätzen. Wann sind diese rechnerisch gefüllt? Ignorieren Sie für diese Näherung die periodischen Bursts. (2 Punkte)
3. Warum schützt eine größere Queue die periodischen Tasks nicht automatisch vor zusätzlicher CPU-Last? Nennen Sie eine begründete Reaktion bei voller Queue. (2 Punkte)
4. Ein normaler FreeRTOS-Thread verarbeitet „höchstens zwei Nachrichten, dann `osDelay(10)`“. Laufzeiten der Nachrichten sind unbekannt. Ist damit ein 2-ms-CPU-Server mit 10-ms-Periode nachgewiesen? Begründen Sie. (2 Punkte)

Aufgabe 5: Noch Arbeit, aber kein Budget

Eine Anfrage wartet im Server. Der Prozessor ist sonst frei, das Budget aber erschöpft. Weshalb kann Leerlauf richtig sein? Welche zusätzliche Annahme wäre für eine garantierte Antwortzeit jeder Anfrage nötig? Unterscheiden Sie außerdem die Warteschlange im Modell von einem Thread, der bloß aktiv seine Zeit prüft.

Aufgabe 6: Einen Polling Server als LED-Modell selbst simulieren

Programmieren, etwa 25–35 Minuten. Verwenden Sie die bekannte **M4-only-FreeRTOS-Basis aus Übung 3** mit **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q**. Nur LD1/PB0 und LD2/PE1 werden benötigt. Die Projektvorbereitung nennt die CubeMX-Schritte. Die Datenstruktur und Implementierungsregeln stehen direkt in dieser Aufgabe.

Ein virtueller Schritt stellt eine Modell-Millisekunde dar. Zur Anzeige führt eine gewöhnliche Simulationstask nur ungefähr alle **100 reale Kernelticks** einen Schritt aus. Diese Verlangsamung ist keine CPU-Budgetmessung und implementiert keinen Scheduling-Server im FreeRTOS-Kernel.

```
typedef struct {
    uint32_t t;
    uint32_t pending;
    uint32_t budget;
    uint32_t served;
} Modell;
```

1. Implementieren Sie `uint32_t ModellSchritt(Modell *m)` für einen gültigen Zeiger. Der Anfangszustand ist in allen Feldern null. Bei den virtuellen Zeiten **1, 3, 7 und 8** kommt jeweils genau ein Auftrag mit einer Einheit Arbeit an. Erhöhen Sie zuerst `pending` für eine Ankunft. Bei `t % 6U == 0U` ist Pollzeit: Setzen Sie `budget` auf zwei, falls Arbeit wartet, sonst auf null. Bei `pending > 0U && budget > 0U` wird genau eine Einheit ausgeführt: beide Werte um eins vermindern, `served` um eins erhöhen, Rückgabewert eins. Sonst Rückgabewert null. Wird die Arbeit leer, verfällt Restbudget. Erhöhen Sie danach `t` um eins. Es gibt im endlichen Versuch weder Zählerüberlauf noch mehr als vier Aufträge.
2. Legen Sie in CubeMX genau zwei CM4-Threads an: `StartHeartbeatTask` mit Normal-Priorität und 512 Byte Stack, `StartSimulationTask` mit BelowNormal-Priorität und 1024 Byte Stack. Heartbeat schaltet LD2 um und blockiert mit `osDelay(500)`. Die Simulation hält ein lokales `Modell m = {0};`, wiederholt die Schritte 0 bis 17 und beginnt dann mit einem neuen Nullzustand. Sie schreibt ausschließlich LD1: an bei Rückgabewert eins, sonst aus, danach `osDelay(100)`. Prüfen Sie beide Delay-Rückgaben und die vom Generator angelegten Handles.
3. Prüfen Sie Ihre Funktion zuerst auf Papier: In welchen Schritten muss LD1 an sein? Welchen Wert hat `served` nach Schritt 17? Bauen und laden Sie anschließend nur CM4, beobachten Sie das Muster und die unabhängige Heartbeat-LED. Geben Sie den tatsächlichen Prüfstatus an.

Ergebnis: eigener endlicher Modellschritt und zwei laufende Threads. Modellzeit, reale Kernelticks und tatsächliche CPU-Zeit sind drei verschiedene Größen. M7 bleibt deaktiviert, Taster und zusätzliche Timer werden nicht verwendet.